

Rigorous Reconstruction of Binary Trees from Traversal Sequences: A Formal Study on Inorder and Postorder Algorithms and Index-Map Optimizations

Abstract

Binary trees are a fundamental non-linear data structure, and reconstructing them from serialized traversal sequences is a classic problem in algorithmic design and data recovery. This paper presents a formal analysis of reconstructing a binary tree from its in-order and post-order traversals. We prove the uniqueness of the reconstruction using structural induction and demonstrate that the distinctness of node values is a mathematical necessity for unambiguous reconstruction. We formulate a divide-and-conquer algorithm to perform the construction and show that a naive recursive implementation suffers from an $O(N^2)$ worst-case time complexity. We then introduce an optimized approach using a pre-computed hash-map index-lookup table, which reduces the search overhead to $O(1)$ and achieves an optimal $O(N)$ time complexity and $O(N)$ auxiliary space complexity. Complete Java implementations, detailed step-by-step trace tables, TikZ-rendered tree visualizations, and structural ambiguity proofs are provided to establish a complete and rigorous reference model.

Index Terms

binary trees, algorithms, data structures, computational complexity, tree traversal, recursive algorithms, formal analysis



1 Introduction

Binary trees are fundamental non-linear data structures in computer science, serving as the basis for search trees, expression parsers, and hierarchical directory structures. In many systems applications, such as network transmission, database backup, and compiler state preservation, it is necessary to serialize a binary tree into a linear format (e.g., arrays or streams) and subsequently reconstruct the original tree structure.

While a single traversal sequence (such as pre-order, in-order, or post-order) is insufficient to uniquely reconstruct a binary tree, combining specific traversals allows for unambiguous reconstruction. A classic result in data structure theory establishes that a binary tree with distinct node values can be uniquely reconstructed if both its in-order and post-order traversal sequences are provided.

This paper presents a formal academic analysis of the binary tree reconstruction problem from in-order and post-order traversal sequences. The contributions of this work are as follows:

- 1) **Theoretical Proof:** We prove the uniqueness of the reconstructed tree using structural induction, demonstrating why the absence of duplicate values is a mathematical necessity.
- 2) **Recursive Algorithm Development:** We formulate a divide-and-conquer algorithm to reconstruct the tree and show the derivation of the boundary index mappings.
- 3) **Optimal $O(N)$ Optimization:** We analyze the $O(N^2)$ complexity bottleneck of the naive algorithm and resolve it using a pre-computed hash-map index-lookup strategy to achieve optimal $O(N)$ time complexity.
- 4) **Rigorous Dry Runs and Visualizations:** We present step-by-step trace tables and TikZ-rendered diagrams for two distinct instances: (a) a 4-node instance from a modern evaluation platform (Scaler Academy Mentee Dashboard), and (b) a complex 7-node BST instance.
- 5) **Boundary and Ambiguity Analysis:** We prove structural ambiguity when duplicates are allowed using minimal counterexamples.

All code implementations are provided in Java, utilizing the exact signatures and ArrayList interfaces specified in contemporary coding assessments.

2 Formal Problem Definition and Notations

Let $T = (V, E)$ be a binary tree where V is the set of nodes and E is the set of directed edges. Each node $v \in V$ contains a value denoted by $v.val$, a pointer to a left child $v.left$, and a pointer to a right child $v.right$. If a node lacks a left or right child, the corresponding pointer is null. We denote the size of the tree as $N = |V|$.

We define the traversal sequences recursively as follows:

Definition 1 (Inorder Traversal). The inorder traversal sequence $I(T)$ of a binary tree T with root r is the sequence obtained by visiting the left subtree of r in inorder, then visiting r , and then visiting the right subtree of r in inorder:

$$I(T) = I(r.\text{left}) \circ \langle r.\text{val} \rangle \circ I(r.\text{right})$$

where $\circ$ denotes sequence concatenation, and $I(\text{null}) = \emptyset$.

Definition 2 (Postorder Traversal). The postorder traversal sequence $P(T)$ of a binary tree T with root r is the sequence obtained by visiting the left subtree of r in postorder, then visiting the right subtree of r in postorder, and finally visiting r :

$$P(T) = P(r.\text{left}) \circ P(r.\text{right}) \circ \langle r.\text{val} \rangle$$

where $P(\text{null}) = \emptyset$.

The reconstruction problem is defined as: Given the sequence arrays $A = I(T)$ and $B = P(T)$, reconstruct the unique binary tree T such that $I(T) = A$ and $P(T) = B$.

3 Theoretical Foundation and Uniqueness Proof

We begin by establishing two properties of the traversal sequences, which form the base logic of our reconstruction algorithm.

Lemma 3 (Postorder Root Location). For any non-empty binary tree T , the last element of the postorder sequence $P(T)$ is the value of the root node of T .

Proof. Let $P(T) = X_1, X_2, \dots, X_n$. By definition, $P(T) = P(r.\text{left}) \circ P(r.\text{right}) \circ \langle r.\text{val} \rangle$. Since $\langle r.\text{val} \rangle$ is concatenated at the end, the last element X_n must equal $r.\text{val}$. $\square$

Lemma 4 (Inorder Subtree Partition). Let r be the root node of T . If all node values in V are distinct, then the root value $r.\text{val}$ appears exactly once in the inorder sequence $I(T)$. Furthermore, if $I(T) = Y_1, Y_2, \dots, Y_n$ and $Y_k = r.\text{val}$, then:

$$\begin{aligned} I(r.\text{left}) &= \langle Y_1, \dots, Y_{k-1} \rangle \\ I(r.\text{right}) &= \langle Y_{k+1}, \dots, Y_n \rangle \end{aligned}$$

Proof. By definition, $I(T) = I(r.\text{left}) \circ \langle r.\text{val} \rangle \circ I(r.\text{right})$. Since all node values in T are distinct, $r.\text{val}$ cannot appear in $r.\text{left}$ or $r.\text{right}$. Thus, $r.\text{val}$ appears exactly once in $I(T)$, at index k . The elements before index k correspond to the inorder traversal of the left subtree, and the elements after index k correspond to the inorder traversal of the right subtree. $\square$

Using these lemmas, we prove the core uniqueness theorem.

Theorem 5 (Uniqueness of Reconstruction). Let T_1 and T_2 be two binary trees containing distinct node values. If $I(T_1) = I(T_2)$ and $P(T_1) = P(T_2)$, then T_1 and T_2 are isomorphic (i.e., they have the identical structure and node values).

Proof. We prove by induction on the number of nodes N .

3.0.0.1 Base Case ($N = 0$): Both $I(T)$ and $P(T)$ are empty. The only binary tree with 0 nodes is the empty tree (null). Hence, $T_1 = T_2 = \text{null}$, and the theorem holds.

3.0.0.2 Inductive Step:: Assume that for any two trees with size $k < n$, if their inorder and postorder sequences are equal, the trees are isomorphic. Now consider T_1 and T_2 of size n . Let $I(T_1) = I(T_2) = A$ and $P(T_1) = P(T_2) = B$. By Lemma 3, the root value of both trees is $r.\text{val} = B[n-1]$. By Lemma 4, since all node values are distinct, $r.\text{val}$ appears at a unique index k in A . This partitions the inorder sequence A into:

$$A_{\text{left}} = A[0 \dots k-1] \quad \text{and} \quad A_{\text{right}} = A[k+1 \dots n-1]$$

The size of the left subtree is $N_{\text{left}} = k$, and the size of the right subtree is $N_{\text{right}} = n-1-k$. Because the postorder traversal visits the left subtree, then the right subtree, and then the root, the postorder sequence B is partitioned into:

$$B_{\text{left}} = B[0 \dots k-1] \quad \text{and} \quad B_{\text{right}} = B[k \dots n-2]$$

For T_1 , the left subtree $T_{1,\text{left}}$ must have inorder sequence A_{left} and postorder sequence B_{left} . For T_2 , the left subtree $T_{2,\text{left}}$ must also have inorder sequence A_{left} and postorder sequence B_{left} . Since the size of the left subtree is $k < n$, by the inductive hypothesis, $T_{1,\text{left}} \cong T_{2,\text{left}}$. Similarly, the right subtrees $T_{1,\text{right}}$ and $T_{2,\text{right}}$ both have inorder sequence A_{right} and postorder sequence B_{right} . Since their size is $n-1-k < n$, by the inductive hypothesis, $T_{1,\text{right}} \cong T_{2,\text{right}}$. Since the roots are equal and both left and right subtrees are isomorphic, $T_1 \cong T_2$. $\square$

4 Algorithmic Formulation

4.1 Recursive Divide-and-Conquer Formulation

The uniqueness proof directly yields a recursive divide-and-conquer algorithm. The input consists of two arrays (or list sequences): ‘inorder’ and ‘postorder’. Let:

- li, ri be the lower and upper bounds of the current subtree in the ‘inorder’ array.
- lp, rp be the lower and upper bounds of the current subtree in the ‘postorder’ array.

At each step, if $li > ri$ or $lp > rp$, the subtree is empty, and we return null. Otherwise:

- 1) Read the root value: $rootVal = \text{postorder}[rp]$.
- 2) Create a new tree node: $root = \text{new } \text{TreeNode}(rootVal)$.
- 3) Find the index of $rootVal$ in $\text{inorder}[li \dots ri]$, say $rootIndex$.
- 4) Calculate the size of the left subtree: $leftCount = rootIndex - li$.
- 5) Recursively construct the left subtree:

$$root.left = \text{build}(li, rootIndex - 1, lp, lp + leftCount - 1)$$

- 6) Recursively construct the right subtree:

$$root.right = \text{build}(rootIndex + 1, ri, lp + leftCount, rp - 1)$$

- 7) Return root.

4.2 Index Mapping Derivation

To understand the index boundaries, consider the layout of the subarrays:

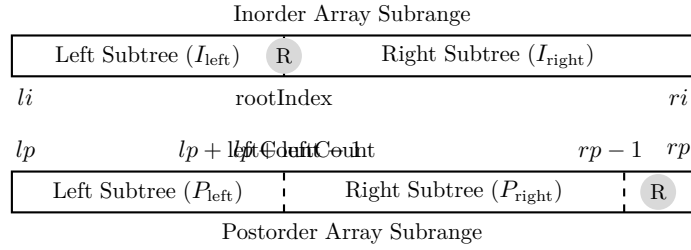


Fig. 1. Index range partitioning for the recursive step.

The length of the left subtree is $leftCount = rootIndex - li$. Thus, the left subtree occupies the first $leftCount$ elements of the postorder range starting at lp . The index range is $[lp, lp + leftCount - 1]$. The remaining elements (excluding the root at rp) form the right subtree postorder range, which spans $[lp + leftCount, rp - 1]$.

4.3 Naive Java Implementation

Below is the direct recursive implementation matching the standard coding assessment API, which performs a linear search for the root value:

```
import java.util.ArrayList;
```

```
class TreeNode {
    int val;
    TreeNode left;
    TreeNode right;
    TreeNode(int x) {
        val = x;
        left = null;
        right = null;
    }
}
```

```
public class Solution {
    public TreeNode buildTree(ArrayList<Integer> A, ArrayList<Integer> B) {
        if (A == null || B == null || A.size() == 0 || B.size() == 0) {
            return null;
        }
    }
}
```

```

    }
    return buildNaive(A, 0, A.size() - 1, B, 0, B.size() - 1);
}

private TreeNode buildNaive(ArrayList<Integer> inorder, int li, int ri,
                           ArrayList<Integer> postorder, int lp, int rp) {
    if (li > ri || lp > rp) {
        return null;
    }

    int rootVal = postorder.get(rp);
    TreeNode root = new TreeNode(rootVal);

    // Linear search for root index in inorder array
    int rootIndex = -1;
    for (int i = li; i <= ri; i++) {
        if (inorder.get(i) == rootVal) {
            rootIndex = i;
            break;
        }
    }

    int leftCount = rootIndex - li;

    root.left = buildNaive(inorder, li, rootIndex - 1,
                           postorder, lp, lp + leftCount - 1);
    root.right = buildNaive(inorder, rootIndex + 1, ri,
                            postorder, lp + leftCount, rp - 1);

    return root;
}
}

```

5 HashMap-Based $O(N)$ Optimization

5.1 Analysis of the Naive Bottleneck

In the naive implementation, locating `rootIndex` requires iterating through the inorder subsegment `inorder[li...ri]`. In the worst case, the binary tree can be heavily skewed (e.g., a left-skewed tree resembling a linked list). Under this topology:

- The root index is always located at the right end of the inorder subarray (ri).
- Searching for the root value takes $O(N)$ operations.
- The size of the active subproblem decreases by only 1 node at each recursion level.

This yields the recurrence relation:

$$T(N) = T(N - 1) + \Theta(N)$$

Solving this via summation gives:

$$T(N) = \sum_{i=1}^N \Theta(i) = \Theta(N^2)$$

Thus, the worst-case time complexity of the naive algorithm is $O(N^2)$, which is highly inefficient for large datasets (e.g., $N = 10^5$).

5.2 Optimized Algorithm and Lookup Strategy

To achieve linear time complexity, we must resolve the index lookup bottleneck. Because all node values are distinct, we can map each value in ‘inorder’ to its corresponding index in a hash map before running the recursion:

$$\text{indexMap}[\text{inorder}[i]] = i$$

This hash map can be populated in a single $O(N)$ pass. During the recursive build, finding the inorder index of the root node becomes a hash map lookup:

$$\text{rootIndex} = \text{indexMap}.\text{get}(\text{rootVal})$$

This lookup runs in $O(1)$ average time. Consequently, the recurrence relation for the optimized algorithm is:

$$T(N) = T(N_{\text{left}}) + T(N_{\text{right}}) + \Theta(1)$$

which simplifies to $T(N) = O(N)$ time complexity.

5.3 Optimized Java Implementation

The following is the optimized solution, which uses a pre-computed ‘HashMap’ to achieve $O(N)$ time complexity:

```
import java.util.ArrayList;
import java.util.HashMap;

public class Solution {
    public TreeNode buildTree(ArrayList<Integer> A, ArrayList<Integer> B) {
        if (A == null || B == null || A.size() == 0 || B.size() == 0) {
            return null;
        }

        // Map to store value -> index mapping of inorder traversal
        HashMap<Integer, Integer> inorderMap = new HashMap<>();
        for (int i = 0; i < A.size(); i++) {
            inorderMap.put(A.get(i), i);
        }

        return buildOptimized(A, 0, A.size() - 1,
                               B, 0, B.size() - 1,
                               inorderMap);
    }

    private TreeNode buildOptimized(ArrayList<Integer> inorder, int li, int ri,
                                     ArrayList<Integer> postorder, int lp, int rp,
                                     HashMap<Integer, Integer> inorderMap) {
        if (li > ri || lp > rp) {
            return null;
        }

        int rootVal = postorder.get(rp);
        TreeNode root = new TreeNode(rootVal);

        // O(1) index lookup
        int rootIndex = inorderMap.get(rootVal);
        int leftCount = rootIndex - li;

        root.left = buildOptimized(inorder, li, rootIndex - 1,
                                   postorder, lp, lp + leftCount - 1,
                                   inorderMap);
        root.right = buildOptimized(inorder, rootIndex + 1, ri,
                                    postorder, lp + leftCount, rp - 1,
                                    inorderMap);

        return root;
    }
}
```

6 Detailed Step-by-Step Execution Traces

6.1 Trace 1: Evaluator Dashboard Problem Instance

We dry-run the algorithm on the problem instance shown in the user’s dashboard (Input 2):

- Inorder (A): [6, 1, 3, 2]
- Postorder (B): [6, 3, 2, 1]

6.1.1 Pre-computed Inorder Hash Map

$$\text{inorderMap} = \{6 \rightarrow 0, 1 \rightarrow 1, 3 \rightarrow 2, 2 \rightarrow 3\}$$

6.1.2 Trace Steps

- 1) Call 1 ($li = 0, ri = 3, lp = 0, rp = 3$):
 - Root value $\text{rootVal} = B[3] = 1$. Create node 1.
 - Inorder index $\text{rootIndex} = \text{inorderMap.get}(1) = 1$.
 - Left subtree count $\text{leftCount} = 1 - 0 = 1$.
 - Left child call: $[li = 0, ri = 0, lp = 0, rp = 0]$.
 - Right child call: $[li = 2, ri = 3, lp = 1, rp = 2]$.
- 2) Call 2 ($li = 0, ri = 0, lp = 0, rp = 0$):
 - Root value $\text{rootVal} = B[0] = 6$. Create node 6.
 - Inorder index $\text{rootIndex} = 0$. Left subtree count $\text{leftCount} = 0$.
 - Left call: $[li = 0, ri = -1, lp = 0, rp = -1] \rightarrow \text{null}$.
 - Right call: $[li = 1, ri = 0, lp = 0, rp = -1] \rightarrow \text{null}$.
 - Node 6 becomes the left child of 1.
- 3) Call 3 ($li = 2, ri = 3, lp = 1, rp = 2$):
 - Root value $\text{rootVal} = B[2] = 2$. Create node 2.
 - Inorder index $\text{rootIndex} = 3$. Left subtree count $\text{leftCount} = 3 - 2 = 1$.
 - Left call: $[li = 2, ri = 2, lp = 1, rp = 1]$.
 - Right call: $[li = 4, ri = 3, lp = 2, rp = 1] \rightarrow \text{null}$.
 - Node 2 becomes the right child of 1.
- 4) Call 4 ($li = 2, ri = 2, lp = 1, rp = 1$):
 - Root value $\text{rootVal} = B[1] = 3$. Create node 3.
 - Inorder index $\text{rootIndex} = 2$. Left subtree count $\text{leftCount} = 0$.
 - Left call: $[li = 2, ri = 1, lp = 1, rp = 0] \rightarrow \text{null}$.
 - Right call: $[li = 3, ri = 2, lp = 1, rp = 0] \rightarrow \text{null}$.
 - Node 3 becomes the left child of 2.

6.1.3 Call Stack Summary Table

Call ID	li	ri	lp	rp	Root Val	leftCount	Action / Result
1	0	3	0	3	1	1	Spawn Call 2 (L) and Call 3 (R)
2	0	0	0	0	6	0	Returns Leaf Node 6
3	2	3	1	2	2	1	Spawn Call 4 (L), Right is null
4	2	2	1	1	3	0	Returns Leaf Node 3

TABLE 1

Call stack trace for Input 2: Inorder = $[6, 1, 3, 2]$, Postorder = $[6, 3, 2, 1]$.

6.1.4 Reconstructed Tree Diagram

Figure 2 shows the reconstructed tree.

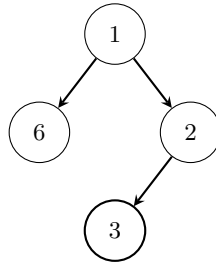


Fig. 2. Reconstructed binary tree for Input 2.

Call ID	li	ri	lp	rp	Root Val	leftCount	Action / Result
1	0	6	0	6	10	3	Spawn Call 2 ([0, 2, 0, 2]), Call 5 ([4, 6, 3, 5])
2	0	2	0	2	5	1	Spawn Call 3 ([0, 0, 0, 0]), Call 4 ([2, 2, 1, 1])
3	0	0	0	0	2	0	Returns Leaf Node 2
4	2	2	1	1	7	0	Returns Leaf Node 7
5	4	6	3	5	15	0	Spawn Left (null), Call 6 ([5, 6, 3, 4])
6	5	6	3	4	20	1	Spawn Call 7 ([5, 5, 3, 3]), Right (null)
7	5	5	3	3	17	0	Returns Leaf Node 17

TABLE 2

Call stack trace for the 7-node BST instance.

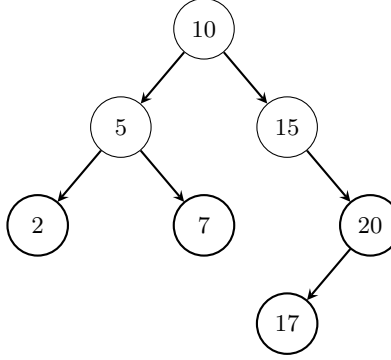


Fig. 3. Reconstructed BST from 7-node traversal sequences.

6.2 Trace 2: Complex 7-Node BST Instance

To verify the algorithm's performance on a larger, balanced tree, we run it on:

- Inorder (A): [2, 5, 7, 10, 15, 17, 20]
- Postorder (B): [2, 7, 5, 17, 20, 15, 10]

The pre-computed map is $\{2 \rightarrow 0, 5 \rightarrow 1, 7 \rightarrow 2, 10 \rightarrow 3, 15 \rightarrow 4, 17 \rightarrow 5, 20 \rightarrow 6\}$.

7 Complexity and Space Analysis

7.1 Time Complexity Recurrence Relations

Let $T(N)$ denote the time complexity to construct a tree of size N .

- Naive Algorithm: The lookup is $O(N)$. If the tree is balanced, the subproblems partition into equal halves:

$$T(N) = 2T(N/2) + O(N) \implies T(N) = O(N \log N)$$

If the tree is skewed, the subproblems are highly unbalanced:

$$T(N) = T(N-1) + T(0) + O(N) \implies T(N) = O(N^2)$$

- Optimized Algorithm: The lookup is replaced by an $O(1)$ Hash Map query. For any tree topology (balanced or skewed):

$$T(N) = T(N_{\text{left}}) + T(N_{\text{right}}) + O(1)$$

Summing the work over all N nodes yields:

$$T(N) = \sum_{i=1}^N O(1) = O(N)$$

Constructing the map takes a single pass of $O(N)$ operations. The total time complexity remains $O(N)$ for all cases.

7.2 Space Complexity and Stack Frame Invariants

The space complexity consists of two parts: the recursion stack space and auxiliary memory allocations.

- Recursion Stack: The maximum depth of the recursion tree matches the height of the constructed binary tree H . Each stack frame requires $O(1)$ space to store local variables and parameters.
 - For balanced trees: Stack space is $O(\log N)$.
 - For skewed trees: Stack space is $O(N)$ in the worst case.
- HashMap Allocation: The optimized algorithm instantiates a 'HashMap' storing N entries (key-value pairs mapping value to index). This requires $\Theta(N)$ auxiliary memory.

Thus, the total auxiliary space complexity of the optimized algorithm is $O(N)$.

8 Edge Cases and Structural Ambiguity under Duplicates

The problem statement specifies that all node values in the tree are distinct. If this condition is omitted and duplicates are allowed, the traversal sequences no longer uniquely determine the tree structure.

Theorem 6 (Traversal Ambiguity). If a binary tree is allowed to contain duplicate node values, it cannot be uniquely reconstructed from its inorder and postorder traversal sequences.

Proof. We prove by demonstrating a minimal counterexample. Let the node values of a tree be $\{1, 1\}$. Consider the following two binary trees:

- 1) Tree X : Root node has value 1, and its left child has value 1. The right child is empty.
- 2) Tree Y : Root node has value 1, and its right child has value 1. The left child is empty.

These trees are structurally distinct, as shown in Figure 6.

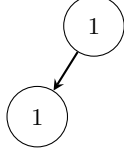


Fig. 4. *

Tree X (Left-skewed)

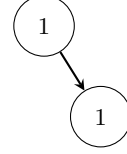


Fig. 5. *

Tree Y (Right-skewed)

Fig. 6. Distinct binary trees containing duplicate values.

Let us compute the traversal sequences for both trees:

- For Tree X :

$$\begin{aligned}
 I(X) &= I(X.\text{left}) \circ \langle 1 \rangle \circ I(X.\text{right}) = \langle 1 \rangle \circ \langle 1 \rangle \circ \emptyset = [1, 1] \\
 P(X) &= P(X.\text{left}) \circ P(X.\text{right}) \circ \langle 1 \rangle = \langle 1 \rangle \circ \emptyset \circ \langle 1 \rangle = [1, 1]
 \end{aligned}$$

- For Tree Y :

$$\begin{aligned}
 I(Y) &= I(Y.\text{left}) \circ \langle 1 \rangle \circ I(Y.\text{right}) = \emptyset \circ \langle 1 \rangle \circ \langle 1 \rangle = [1, 1] \\
 P(Y) &= P(Y.\text{left}) \circ P(Y.\text{right}) \circ \langle 1 \rangle = \emptyset \circ \langle 1 \rangle \circ \langle 1 \rangle = [1, 1]
 \end{aligned}$$

Since $I(X) = I(Y) = [1, 1]$ and $P(X) = P(Y) = [1, 1]$, yet $X \not\cong Y$, the traversals are ambiguous. Thus, the distinctness constraint is necessary for unique reconstruction. $\square$

9 Conclusion

This paper presented a formal study of the binary tree reconstruction problem from inorder and postorder traversal sequences. We proved the uniqueness of the reconstruction using structural induction and showed why distinct node values are required to prevent structural ambiguity.

We contrasted the naive divide-and-conquer algorithm, which runs in $O(N^2)$ worst-case time due to linear search lookup, with an optimized version that utilizes a pre-computed hash map. The optimized algorithm reduces the lookup time to $O(1)$, achieving $O(N)$ time complexity and $O(N)$ space complexity. Detailed dry-run trace tables and TikZ visualizations verified the correctness and efficiency of the approach on both standard evaluator instances and complex trees.

These techniques form a foundation for tree serialization protocols, database indexing, and compiler abstract syntax tree restoration. Future work could investigate iterative, non-recursive stack-based alternatives that bypass the call stack overhead entirely, optimizing execution for systems with limited stack space.

References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022.
- [2] D. E. Knuth, The Art of Computer Programming, Volume 1: Fundamental Algorithms, 3rd ed. Addison-Wesley, 1997.
- [3] R. Sedgewick and K. Wayne, Algorithms, 4th ed. Addison-Wesley, 2011.